

System Card: LFW Face Verification System

MSML/MSAI 605 – Milestone 4 (Final Release v1.0-final)

1. System Overview

What it does. This system is a binary *face-verification* model: given two face images, it decides whether they belong to the *same person* (label 1) or to *different people* (label 0), and emits a similarity score, a calibrated confidence in $[0.5, 1.0]$, and a wall-clock latency.

Pipeline. The deployed system is a five-stage embedding-based pipeline:

1. **Preprocessing:** open RGB image, resize to 160×160 , normalize to $[-1, 1]$.
2. **Embedding:** FaceNet (InceptionResnetV1, pretrained on VGGFace2) produces a 512-dim vector per image.
3. **Similarity:** vectorized cosine similarity (`src/similarity.py`).
4. **Decision:** $score \geq \theta \Rightarrow same$.
5. **Confidence:** deterministic threshold-distance rule (`src/confidence.py`), 0.5 at the boundary, 1.0 at the score extremes.

Inputs / outputs. Inputs: two image paths (or a CSV batch of pairs). Outputs: `score`, `decision` (`{same, different}`), `confidence`, `latency_ms`.

Interfaces. Importable Python (`src/inference.py::infer_pair`); CLI (`scripts/infer.py`); Dockerized CLI (`Dockerfile`, entry point `python scripts/infer.py`).

2. Intended Use

In scope.

- Educational demonstration of an embedding-based verification pipeline.
- Offline batch verification on LFW-style *cropped, frontal, well-lit* face crops.
- Coursework reproducibility, benchmarking, and ablations.

Out of scope.

- Any biometric, identification, or surveillance deployment.
- Liveness or anti-spoofing decisions; this system does no liveness check.
- Identity re-identification across very different sensor types or domains.
- Decisions about access, hiring, law enforcement, or any high-stakes context.
- Inputs that are not face crops (full scenes, multi-face images, non-face content).

3. Data Summary

Source. Labeled Faces in the Wild (LFW), *deep-funneled* variant, accessed via `kagglehub` (dataset `jessicali9530/lfw-dataset`, version 4).

Splits. Deterministic 70/15/15 train/val/test split with `seed=42`; identities stay within a single split. Pair generation also uses `seed=42`. After filtering unreadable image paths the released system uses 4,199 train, 900 val, and 901 test pairs (see `outputs/pairs/`). *Validation* is the only split used for threshold selection; *test* is held out.

Important data limitations.

- LFW is web-scraped from news photography. It is heavily skewed toward public figures (politicians, athletes, celebrities) and toward adult, frontal, well-lit faces.
- Demographic composition of LFW is uneven and not labeled in this dataset, so the released system carries no reliable per-group metric.
- Some identities dominate the pair set (e.g. `George_W_Bush`). Milestone 2 introduced an optional identity-cap + label-rebalance pass (`outputs/pairs_improved/`); the released system reports metrics on the *original* pair set so the numbers are comparable across milestones.

4. Operating Threshold and Key Metrics

Threshold selection. Sweep cosine-similarity thresholds on the validation split over $[-0.5, 1.0]$ at 200 evenly spaced points; pick the threshold that maximizes **F1** on validation. The threshold is locked before touching test.

Selected operating threshold: $\theta = 0.351759$ (see `outputs/threshold/selected_threshold.json`, also written into `configs/m3.yaml::inference.threshold`).

Split	Accuracy	Precision	Recall	F1
Validation ($N = 900$, 457 pos)	0.9633	0.9670	0.9606	0.9638
Test ($N = 901$, 468 pos)	0.9734	0.9723	0.9765	0.9744

Test confusion matrix ($\theta = 0.351759$): TP=457, FP=13, TN=420, FN=11 (`outputs/eval/test_confusion_matrix`)

Confidence. Calibrated to $[0.5, 1.0]$ from $\frac{|score-\theta|}{range\ to\ extreme}$, so 0.5 means “borderline” and 1.0 means “score is at the extreme of its range.” This is a deterministic mapping with no learned parameters; it does not produce calibrated probabilities of identity.

5. Failure Modes and Limitations

The verifier was observed to be less reliable, or can break, on the following inputs:

- **Visually similar different identities** – e.g. relatives, makeup, or near-look-alikes. Cosine similarity is high; the decision can flip to “same.”
- **Off-pose / non-frontal faces** – LFW is largely frontal; profile and extreme yaw inputs are out-of-distribution and produce noisier scores.

- **Strong lighting or low-quality crops** – under/overexposed images, motion blur, low resolution below the 160×160 expected size.
- **Partial occlusion** – masks, sunglasses, hands, hair covering the face.
- **Different sensor / domain** – non-RGB, infrared, heavily filtered, or cartoon images are not in the training distribution and outputs should not be trusted.
- **Non-face inputs** – the pipeline does not detect or reject non-face content; it will silently produce a similarity score.
- **Dataset-specific bias** – LFW is overrepresented by adult public figures; performance on under-represented populations is not characterized here and should be assumed worse than the headline numbers.

6. Fairness-Risk Discussion

This system uses a public dataset (LFW) without reliable demographic metadata, so we do *not* make subgroup performance claims. We instead document risk categories that peer-reviewed face-recognition literature (e.g. NIST FRVT, Buolamwini & Gebru 2018) has repeatedly shown to drive uneven accuracy:

- **Population coverage:** LFW skews white, male, and adult. A pre-trained VGGFace2 backbone was trained on a similarly biased web corpus. Performance on women, on darker-skinned individuals, and on children should be expected to be *worse* than the aggregate metrics in this card.
- **Image-quality differences:** image quality often correlates with demographics (e.g. camera availability, lighting in source photos). Score distributions may therefore diverge along uneven lines even when the model is “the same.”
- **Threshold selection:** the operating threshold is tuned to maximize aggregate F1 on validation. A single global threshold can disadvantage subgroups where the score distribution shifts.
- **Deployment misuse:** the system is *not* a re-identification or surveillance tool. Using it as one would amplify any subgroup-level error against the people most affected.

Mitigations in this release. (i) The Out-of-scope section above explicitly forbids high-stakes use. (ii) The system emits a calibrated confidence so callers can apply abstention near θ . (iii) The data-centric improvement script (`scripts/apply_data_centric.py`) demonstrates how to cap dominant identities for fairer evaluation; the metric numbers in this card are reported on the unfiltered pair set so they remain comparable to Milestone 2.

7. Operational Constraints

- **Input format:** RGB face crops; the system internally resizes to 160×160 and normalizes to $[-1, 1]$.
- **Hardware:** tested on CPU only (Apple M-series, 8 logical cores). GPU is supported in code but optional; see profiling report for measurement methodology.
- **Latency / throughput** (CPU baseline, see profiling report): end-to-end median ≈ 124 ms per pair at batch 1; throughput peaks near ≈ 12 pairs/s at batch 8 and degrades sharply at batch ≥ 16 on the 8-core profiling host (likely thermal / memory pressure).

- **Embedding stage dominates runtime** ($\approx 94\%$ of single-pair end-to-end). Preprocessing is $\approx 6\%$; cosine scoring is $< 0.2\%$.
- **Determinism**: all random operations use `seed=42`. The inference path itself has no randomness.
- **Dependencies**: pinned in `requirements.txt`; the `Dockerfile` bakes the FaceNet weights into the image so the container works offline.
- **Concurrency**: 4-worker thread load test from Milestone 3 hit ≈ 22.6 req/s with stable P95 (`outputs/load_test/runtime_summary.json`).

8. Reproducibility Pointer

- **Repository**: this directory; final tag `v1.0-final`.
- **Reproducibility checklist**: `reports/reproducibility_checklist.md`.
- **Profiling report**: `reports/profiling_report.pdf` (sources in `reports/profiling_report.tex`; raw artifacts in `outputs/profiling/`).
- **README**: top-level `README.md` – copy-pastable setup, CLI, Docker, and profiling commands.
- **Final config**: `configs/m3.yaml` (seed, threshold 0.351759, score range, preprocessing).
- **Selected threshold artifact**: `outputs/threshold/selected_threshold.json`.
- **Test metrics artifact**: `outputs/eval/test_metrics.json`.